

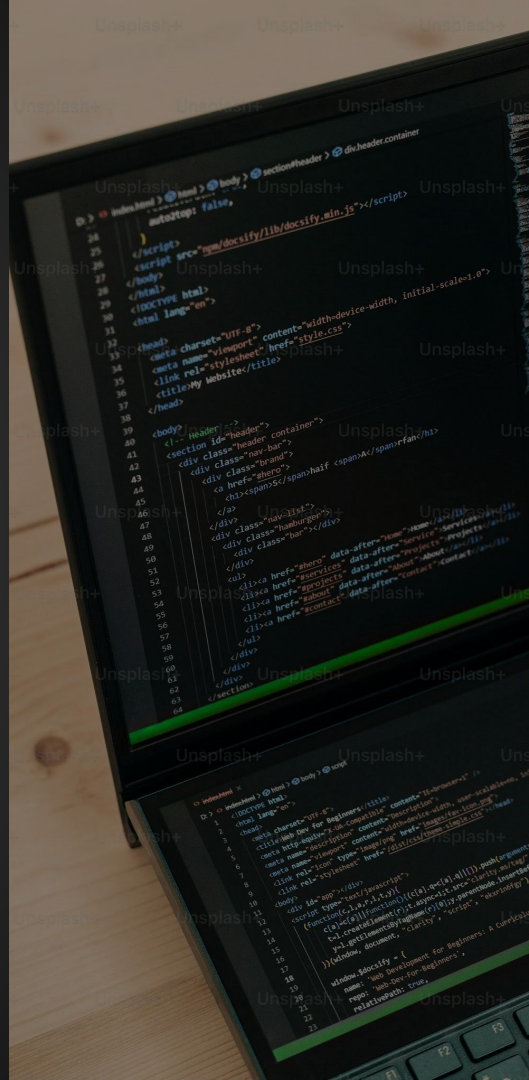
Desenvolvimento de Microserviços

Prof. Guilherme Alves

Comunicação assíncrona

A comunicação assíncrona ocorre quando um serviço envia uma solicitação para outro sem esperar uma resposta imediata.

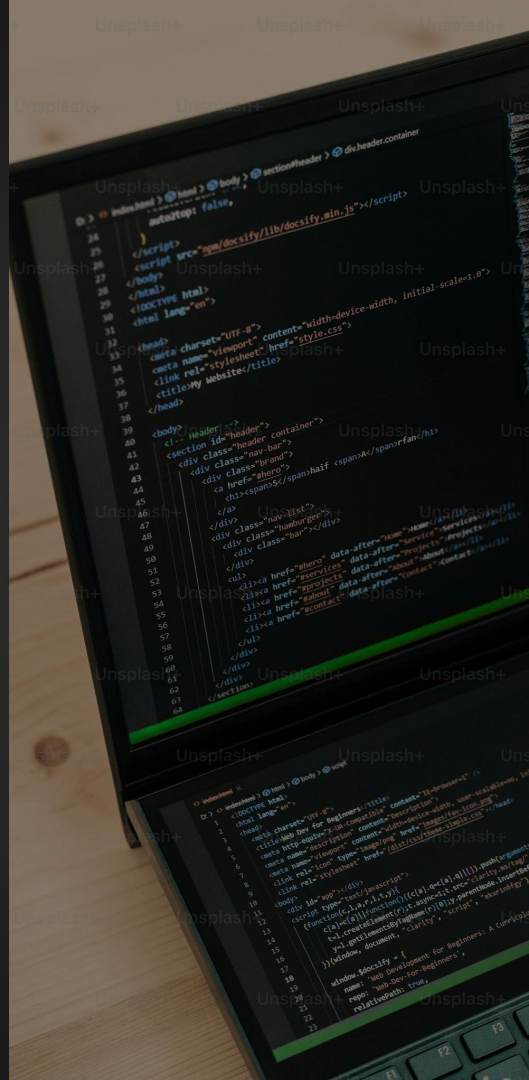
Isso permite que o serviço emissor continue processando outras tarefas enquanto aguarda uma resposta futura (se necessário).



Mensageria

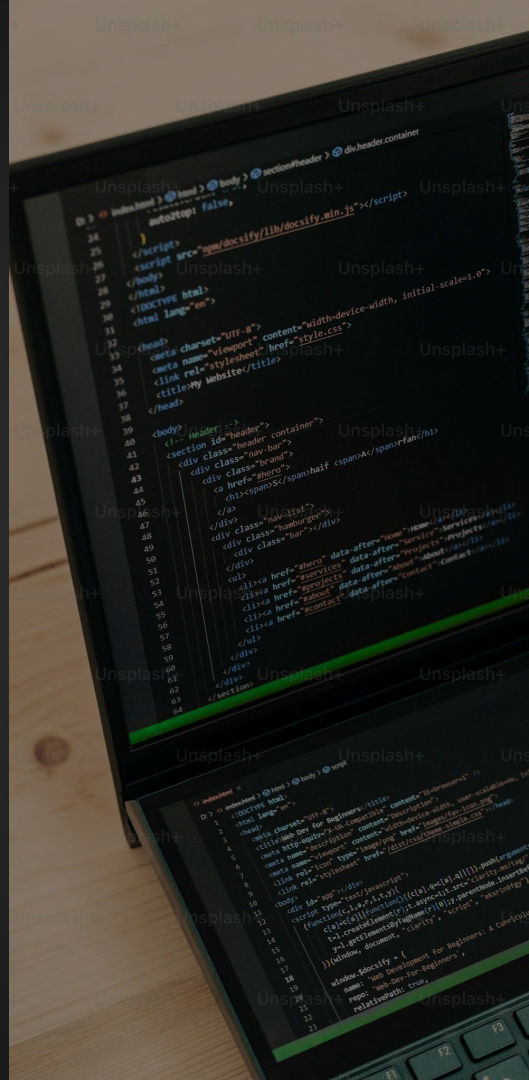
Mensageria é o uso de intermediários (*brokers*) para troca de mensagens entre sistemas.

Um serviço envia mensagens para uma fila ou tópico, e outros serviços podem consumir essas mensagens posteriormente.



Características

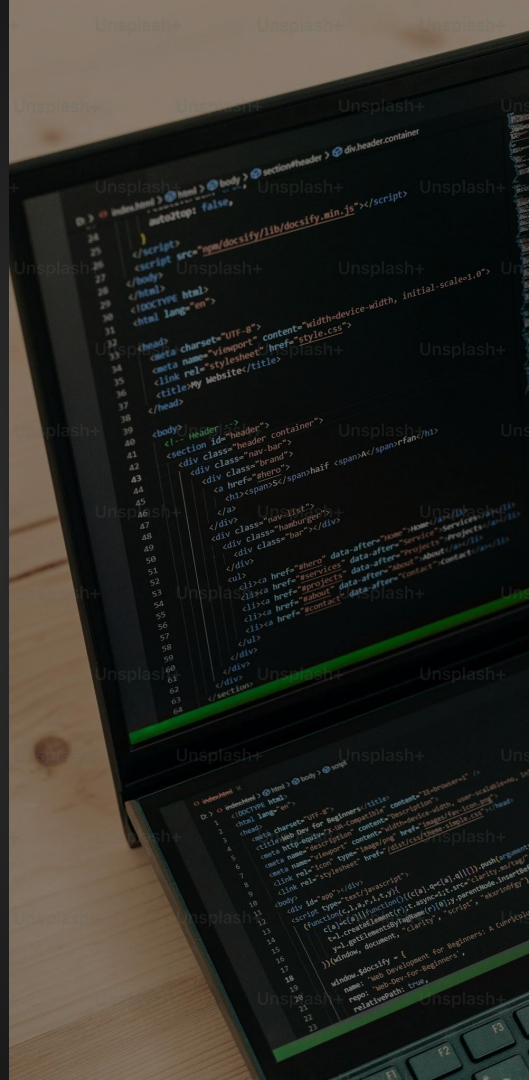
- **Desacoplamento** - Os serviços não precisam conhecer diretamente uns aos outros.
- **Escalabilidade** - Reduz bloqueios e permite o uso eficiente de recursos.
- **Tolerância a falhas** - Se um serviço estiver temporariamente indisponível, a mensagem pode ser armazenada e processada mais tarde.



Como funciona a comunicação?

A comunicação baseada em mensagens geralmente envolve três componentes principais:

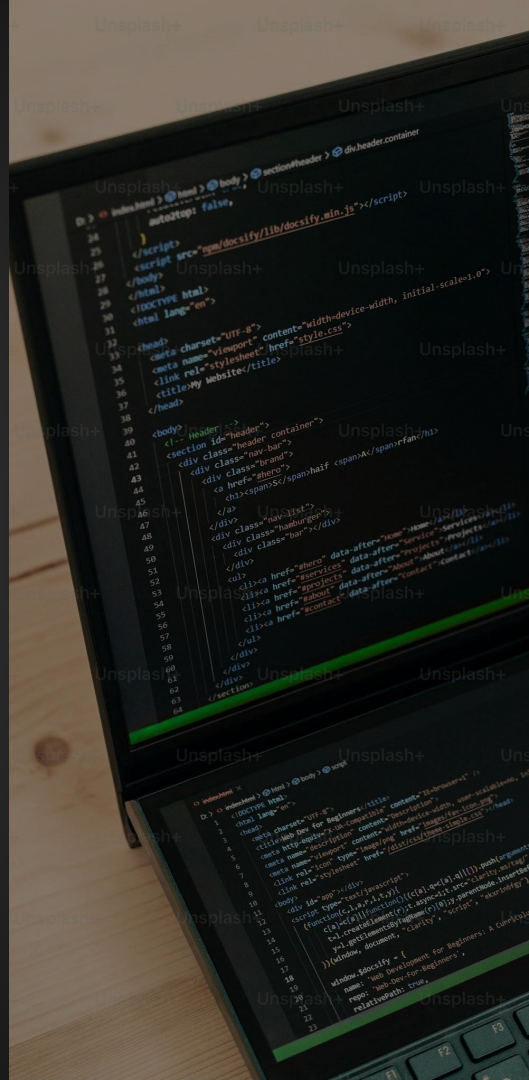
- **Produtor** (*Publisher/Producer*) - Envia mensagens para um broker.
- **Message Broker** - Intermediário que gerencia o armazenamento e a entrega das mensagens.
- **Consumidor** (*Subscriber/Consumer*) - Recebe e processa a mensagem.



O que é um broker?

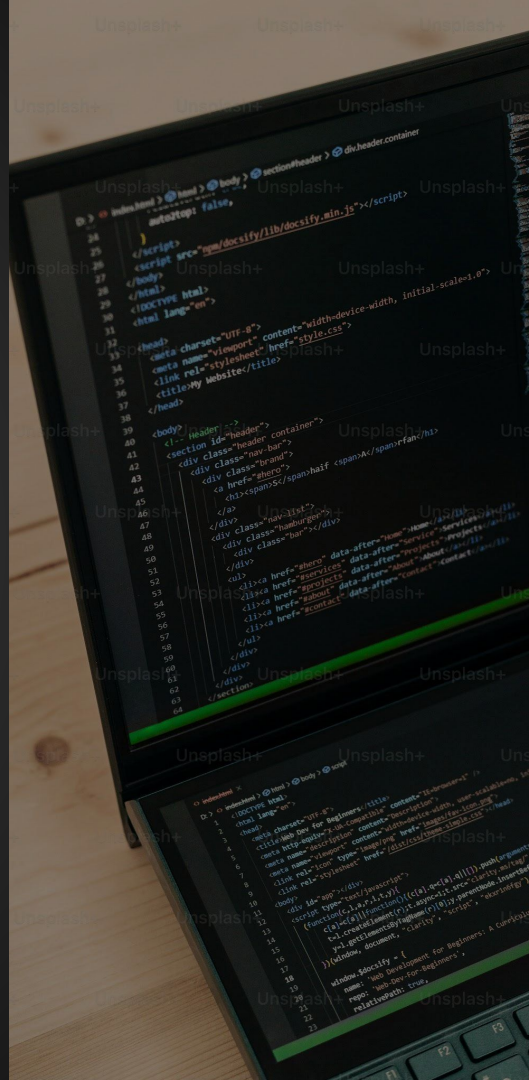
Um **broker** de mensagens é um intermediário de comunicação que gerencia a troca de mensagens entre diferentes sistemas.

Ele serve como uma camada que desacopla os componentes que enviam e recebem mensagens, facilitando a comunicação assíncrona e garantindo que as mensagens sejam entregues de maneira confiável.



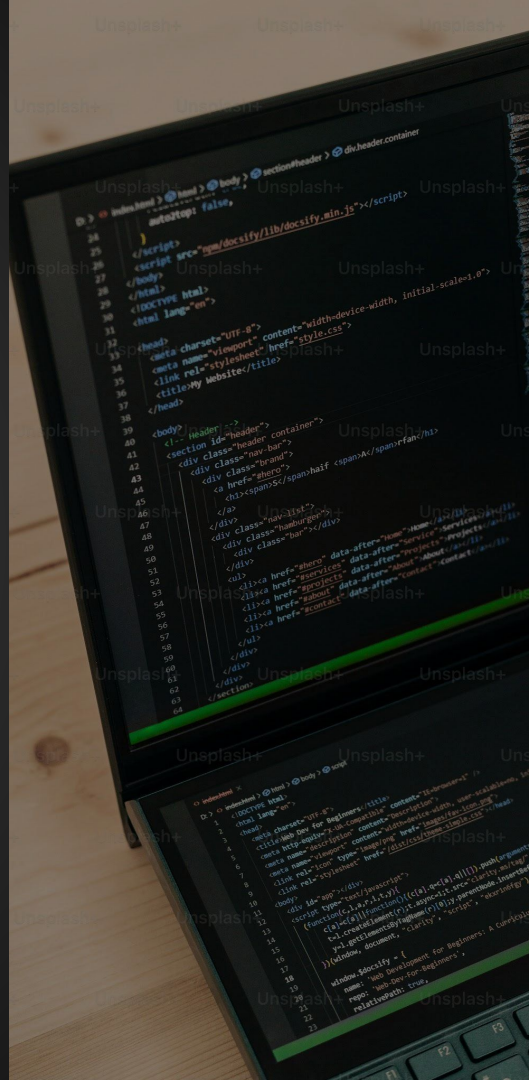
Funções de um broker

- Gerenciamento de filas
- Desacoplamento
- Garantia de entrega
- Escalabilidade



Brokers mals populares

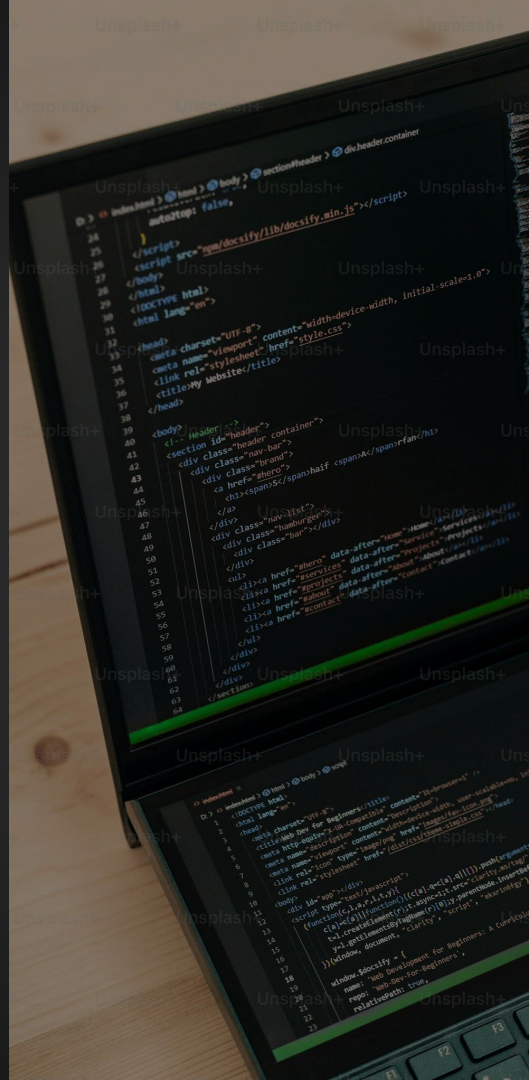
- **RabbitMQ:** Baseado no protocolo AMQP, muito utilizado em sistemas que exigem alta confiabilidade e flexibilidade.
- **Apache Kafka:** Focado em alto desempenho para grandes volumes de dados em tempo real e análise de eventos.
- **ActiveMQ:** Suporta diversos protocolos e é usado em ambientes corporativos, especialmente em sistemas Java.
- **Amazon SQS:** Serviço gerenciado de filas da AWS, ideal para escalabilidade em aplicações na nuvem.



RabbitMQ

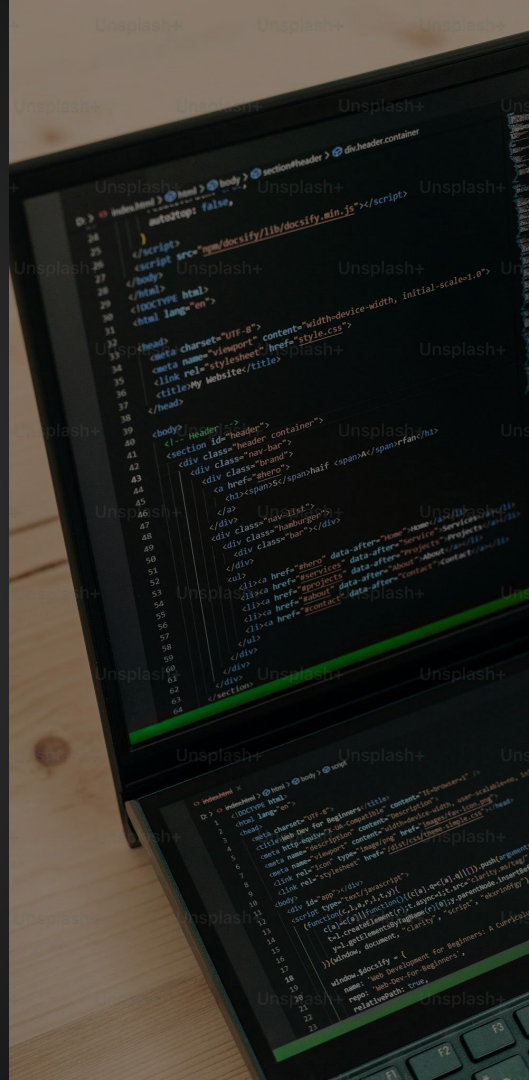
O **RabbitMQ** é um broker de mensagens de código aberto que implementa o protocolo **AMQP** (*Advanced Message Queuing Protocol*).

Ele permite a comunicação assíncrona entre sistemas e é amplamente utilizado para integrar micro-serviços, sistemas distribuídos e aplicações que necessitam de alta confiabilidade na troca de mensagens.



RabbitMQ

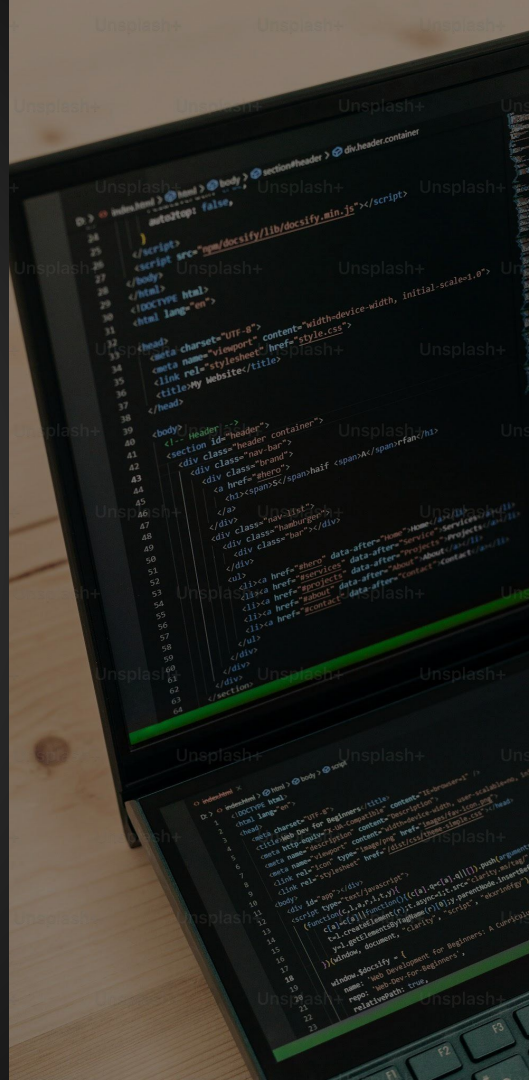
- Fácil de configurar e usar.
- Baixa latência, ideal para aplicações que exigem respostas rápidas.
- Suporte a múltiplos protocolos (*AMQP*, *MQTT*, *STOMP*).
- Escalável e confiável com suporte a clusterização.
- Ferramentas de administração e monitoramento fáceis de usar.



RabbitMQ - Channel

Um **Channel** é uma conexão virtual dentro de uma conexão física com o **RabbitMQ**.

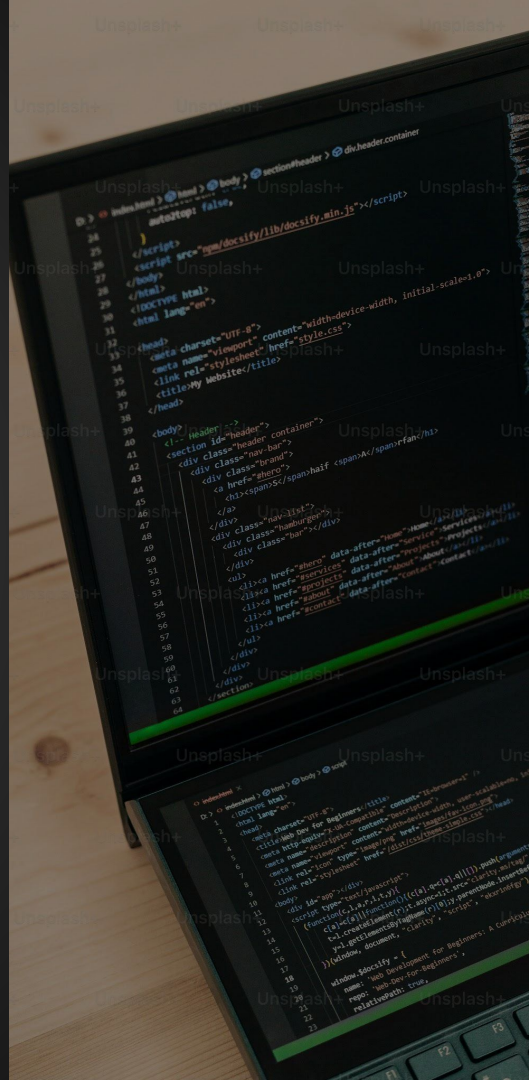
Ele permite que um cliente (produtor ou consumidor) envie e receba mensagens sem precisar abrir várias conexões TCP, o que melhora o desempenho.



RabbitMQ - Exchange

O **Exchange** recebe mensagens dos produtores e decide para quais filas essas mensagens devem ser enviadas.

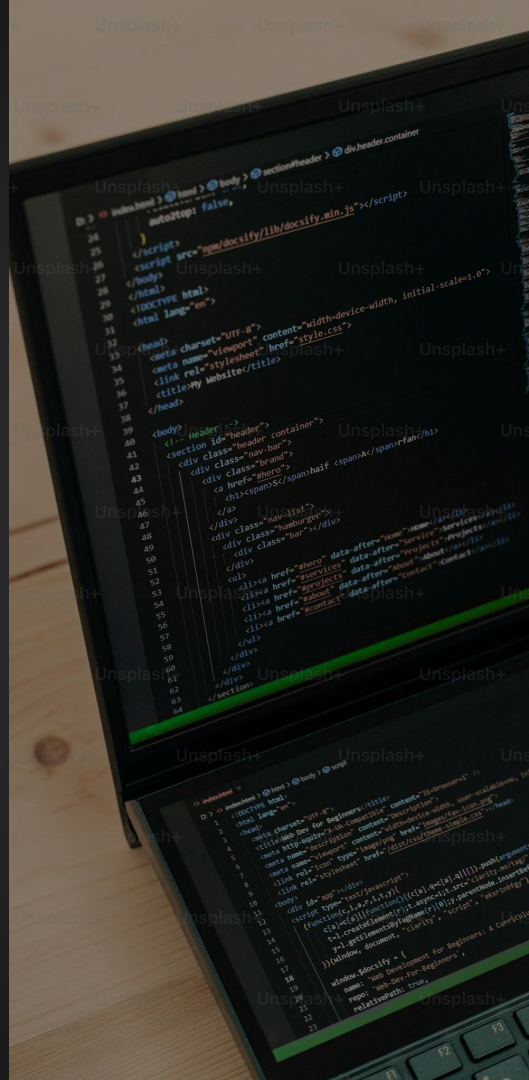
Ele age como um roteador de mensagens.



RabbitMQ - Queue

A **Queue** é onde as mensagens são armazenadas até serem processadas pelos consumidores.

Uma fila pode receber mensagens diretamente de um Exchange, dependendo das regras de roteamento.



Fluxo

1. O **Produtor** envia uma mensagem para um **Exchange**.
2. O **Exchange** determina para qual **Queue** a mensagem deve ser encaminhada.
3. O **Consumidor** recebe e processa a mensagem da **Queue**.

Instalação

<https://www.rabbitmq.com/docs/download>

Docker

```
docker run -d \  
--name rabbitmq \  
-p 5672:5672 \  
-p 15672:15672 \  
-e RABBITMQ_DEFAULT_USER=admin \  
-e RABBITMQ_DEFAULT_PASS=admin \  
rabbitmq:3-management
```